

Week Summary

Accelerators and GPUs

Accelerators and GPUs

- GPUs offer good floating point performance, memory bandwidth and energy efficiency
- A significant fraction of the current Top500 systems use GPUs and GPUs also feature in forthcoming exa-scale systems
- GPU architectures favour data parallelism where different threads execute the same instructions on different data
- The GPU and CPU have separate memory and we need to pay attention to data transfers to get good performance

GPU programming models

- There are a number of different ways to program a GPU
- Some are portable between different hardware, others are vendor specific
- We saw an example of using OpenMP to offload a loop to a GPU (this generated CUDA for an NVIDIA GPU)
- The data movement was optimised by specifying which arrays should be mapped to and from the device

CUDA libraries

Math Libraries



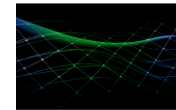
cuBLAS

GPU-accelerated basic linear algebra (BLAS) library



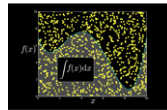
cuFFT

GPU-accelerated library for Fast Fourier Transforms



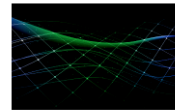
CUDA Math Library

GPU-accelerated standard mathematical function library



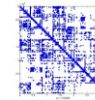
cuRAND

GPU-accelerated random number generation (RNG)



cuSOLVER

GPU-accelerated dense and sparse direct solvers



cuSPARSE

GPU-accelerated BLAS for sparse matrices



cuTENSOR

GPU-accelerated tensor linear algebra library



AmgX

GPU-accelerated linear solvers for simulations and implicit unstructured methods

<https://developer.nvidia.com/gpu-accelerated-libraries>